

REPORT CS 343 LAB 1

Group

- Raj Nandini Kumari : 230123048
- Neha Sellappan Devadas: 230123040
- Subhashree Kedia: 230123063
- Aditya Samal : 230123002

Task 1.1:

Assigning all newly created processes process IDs (PIDs) which are prime numbers.

Step - 1

In the file `proc.c` the following changes were made :

```
int isPrime(int n){
    for(int i=2;i<n;i++){
        if(n%i==0) return 0;
    }
    return 1;
}

void nextPrime(){
    while(!isPrime(nextpid)){
        nextpid++;
    }
}
```

The above 2 functions were made to generate the next prime number using the global variable “nextpid”.

- The function `nextPrime()` generated the next prime number.
- The function `isPrime()` is used to check whether a number is prime or not.

Step - 2

```
int
allocpid()
{
    int pid;
    acquire(&pid_lock);
    pid = nextpid;
    nextpid = nextpid + 1;
    nextPrime();
    printf("Allocated pid: %d\n",pid);
    release(&pid_lock);

    return pid;
}
```

Modified the `allocpid()` function such that the above functions are called.

Instead of assigning `nextpid++` directly, we loop until we find the next prime number.

Step - 3

Created a new file named **task1.c** in user directory in xv6 to make random processes and observe the PIDs allocated.

```
#include "user/user.h"
#include "kernel/types.h"

int main(void)
{
    int pid1 = fork();
    if(pid1 == 0){
        printf("Child 1: PID = %d\n", getpid());
        exit(0);
    }
    wait(0);

    int pid2 = fork();
    if(pid2 == 0){
        printf("Child 2: PID = %d\n", getpid());
        exit(0);
    }
    wait(0);

    int pid3 = fork();
    if(pid3 == 0){
        printf("Child 3: PID = %d\n", getpid());
        exit(0);
    }
    wait(0);
    printf("Parent: PID = %d\n", getpid());
    exit(0);
}
```

Step - 4

In the **Makefile**, we added task1 in **UPROGS** as **\$U/_task1**.

Observation:

During the execution of Task 1, we noticed that the process ID (PID) allocation began from 7. We want to know which process was assigned the earlier PIDs—specifically 2, 3, and 5. So we modified the code to log the PID each time a process was created and realised the initial prime numbers were assigned to a hart (hart 1, hart 2) and during the initialization (**init**).

Result:

- Each newly forked process now receives a **prime number** as its PID.
- Verified correctness by creating multiple processes and observing consecutive prime PIDs.

The following was printed in the terminal when we executed the `task1.c` file.

```
xv6 kernel is booting
Allocated pid: 2
hart 1 starting
hart 2 starting
init: starting sh
Allocated pid: 3
$ task1
Allocated pid: 5
Allocated pid: 7
Child 1: PID = 7
Allocated pid: 11
Child 2: PID = 11
Allocated pid: 13
Child 3: PID = 13
Parent: PID = 5
$
```

Task 1.2:

Implement a user-space utility `top` that displays basic process information: PID, state, CPU ticks, and process name.

Implementation Overview:

1. Kernel-side:

- In `proc.h`, added `uint64 ticks` variable in `struct proc` and also created another struct `uproc`:

```
struct uproc {
    int pid;
    int state;
    uint ticks;
    char name[16];
};
```

- In `proc.c`, we added `p->ticks++` in `scheduler()` function when the state is set to running, and set `p->ticks=0` in `procinit()` and `freeproc()`.

- Added a system call `getprocs()` in `proc.c` to copy process information into an array of `struct uproc`.

```
int
getprocs(struct uproc *up, int max)
{
    int count = 0;

    for (int i = 0; i < NPROC && count < max; i++) {
        struct proc *p = &proc[i];
        acquire(&p->lock);

        if (p->state != UNUSED) {
            up[count].pid = p->pid;
            up[count].state = p->state;
            up[count].ticks = p->ticks;
            safestrcpy(up[count].name, p->name, sizeof(up[count].name));
            count++;
        }

        release(&p->lock);
    }

    return count;
}
```

- In `sysproc.c`, we added a function called `sys_getprocs()`

```
extern int getprocs(struct uproc, int);
uint64 sys_getprocs(void)
{
    uint64 addr;
    int max;
    argaddr(0, &addr);
    argint(1, &max);
    struct uproc ktable[max];
    int n = getprocs(ktable, max);
    if (copyout(myproc()->pagetable, addr, (char *)ktable, sizeof(struct
uproc) * n) < 0)
        return -1;
    return n;
}
```

- In `syscall.h`, we added:

```
#define SYS_getprocs 23
```

- In `syscall.c`, we added:

```
extern uint64 sys_getprocs(void);
```

```
[SYS_getprocs] sys_getprocs, //in the mapping
```

In User Side

- We added `usys.pl`, we added `entry("getprocs");`
- We created a `top.c` file

```
#include "kernel/types.h"
#include "user/user.h"
#include "kernel/param.h"

struct uproc {
    int pid;
    int state;
    uint ticks;
    char name[16];
};

int getprocs(struct uproc *table, int max);
int main(void)
{
    struct uproc table[64];
    int n = getprocs(table, 64);
    if(n < 0){
        printf("top: getprocs failed\n");
        exit(1);
    }
    for(int i = 0; i < n; i++){
        char *s;
        switch(table[i].state){
            case 1: s = "USED"; break;
            case 2: s = "SLEEPING"; break;
            case 3: s = "RUNNABLE"; break;
            case 4: s = "RUNNING"; break;
            case 5: s = "ZOMBIE"; break;
            default: s = "???"; break;
        }
        printf("PID: %d\n", table[i].pid);
        printf("STATE: %s\n", s);
        printf("TICKS: %d\n", table[i].ticks);
        printf("NAME: %s\n", table[i].name);
        printf("\n");
    }
    exit(0);
}
```

}

- In the [Makefile](#), we added top in UPROGS as `$U/_top\`.

Result:

```
xv6 kernel is booting

hart 1 starting
hart 2 starting
init: starting sh
$ top
Process #1
PID: 2
State: SLEEPING
Ticks: 23
Name: init
-----
Process #2
PID: 3
State: SLEEPING
Ticks: 10
Name: sh
-----
Process #3
PID: 5
State: RUNNING
Ticks: 5
Name: top
-----
```